

162140222 黄钰轩

请用C语言+寄存器的方式编程：

【例 5.3】 当按 KEY1 (PD11) 键时, 让 LED1 (PD2) 闪烁, 当按 KEY2 (PD12) 键时让 LED2 (PD3) 闪烁, 当按 KEY3 (PC13) 键时让 LED3 (PD4) 闪烁, 当按 KEY4 (PA1) 键时让 LED4 (PD5) 闪烁, 当同时按下 KEY1 和 KEY2 时, 四个 LED 发光管全亮。

(1) 初始化 GPIO 端口

```
GPIO_Init()
{
    RCC->APB2ENR |= (1 << 2) | (1 << 4) | (1 << 5);
    GPIOD->CRH |= (1 << 14) | (1 << 18);
    GPIOC->CRH |= (1 << 22);
    GPIOA->CRL |= (1 << 6);
    GPIOD->CRL |= (3 << 8) | (3 << 12) | \
                  (3 << 16) | (3 << 20);
    GPIOD->ODR |= (1 << 2) | (1 << 3) | (1 << 4) | (1 << 5);
}
```

(2) 判断按键并做相应显示处理

```
#define LED1(x) ((x) ? (GPIOD->ODR |= (1 << 2)) \
                      : (GPIOD->ODR &= ~(1 << 2)))
#define LED2(x) ((x) ? (GPIOD->ODR |= (1 << 3)) \
                      : (GPIOD->ODR &= ~(1 << 3)))
#define LED3(x) ((x) ? (GPIOD->ODR |= (1 << 4)) \
                      : (GPIOD->ODR &= ~(1 << 4)))
#define LED4(x) ((x) ? (GPIOD->ODR |= (1 << 5)) \
                      : (GPIOD->ODR &= ~(1 << 5)))

uint16_t Key = 0;
```

请用C语言+寄存器的方式编程：

【例 5.3】 当按 KEY1 (PD11) 键时, 让 LED1 (PD2) 闪烁, 当按 KEY2 (PD12) 键时让 LED2 (PD3) 闪烁, 当按 KEY3 (PC13) 键时让 LED3 (PD4) 闪烁, 当按 KEY4 (PA1) 键时让 LED4 (PD5) 闪烁, 当同时按下 KEY1 和 KEY2 时, 四个 LED 发光管全亮。

(2) 判断按键并做相应显示处理

Switch()

{

switch(Key)

{

case 1:

{

LED1(1); /* PD2 = 1 LED1 亮 */

Delay();

LED1(0); /* PD2 = 0 LED1 灭 */

Delay();

LED2(0); LED3(0); LED4(0);

break;

}

case 2:

{

LED2(1); Delay(); LED2(0); Delay(); LED1(0); LED3(0); LED4(0);

break;

}

case 3:

{

LED3(1); Delay(); LED3(0); Delay(); LED2(0); LED1(0); LED4(0);

break;

}

case 4:

{

LED4(1); Delay(); LED4(0); Delay(); LED2(0); LED3(0); LED1(0);

break;

}

case 5:

{

LED1(1); LED2(1); LED3(1); LED4(1); /* 全亮 */

break;

}

default : //default 表示 switch 语句里所有 case 都不成立时要

{

LED1(0); LED2(0); LED3(0); LED4(0); /* 全灭 */

break;

}

}

}

请用C语言+寄存器的方式编程：

【例 5.3】 当按 KEY1 (PD11) 键时, 让 LED1 (PD2) 闪烁, 当按 KEY2 (PD12) 键时让 LED2 (PD3) 闪烁, 当按 KEY3 (PC13) 键时让 LED3 (PD4) 闪烁, 当按 KEY4 (PA1) 键时让 LED4 (PD5) 闪烁, 当同时按下 KEY1 和 KEY2 时, 四个 LED 发光管全亮。

```
main(void)
{
    SystemInit();
    GPIO_Init();
    while (1)
    {
        if ((GPIOID->IDR & (1 << 11)) == 0)
            Key = 1;
        else if ((GPIOID->IDR & (1 << 12)) == 0)
            Key = 2;
        else if ((GPIOID->IDR & (1 << 1)) == 0)
            Key = 3;
        else if ((GPIOID->IDR & (1 << 13)) == 0)
            Key = 4;
        else if ((GPIOID->IDR & (3 << 11)) == 0)
            Key = 5;
        Delay(); // 后续与课本中一致
        Switch();
    }
}
```

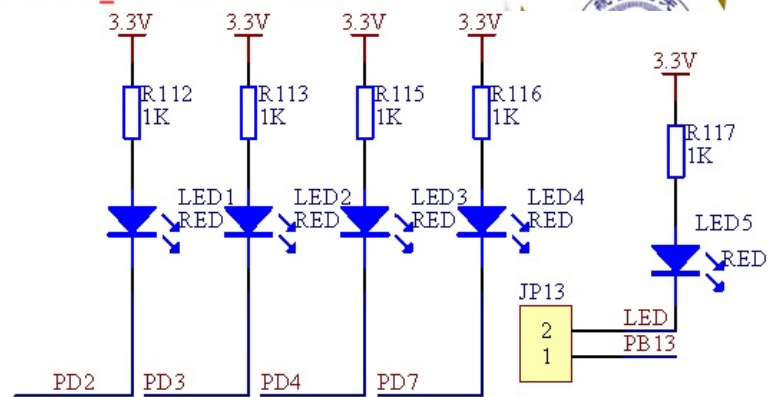
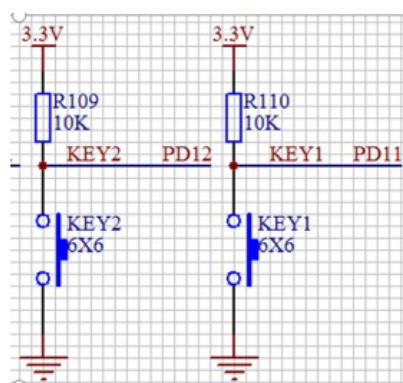
按图示连接，分别用

- (1) 用汇编语言操作方式
- (2) 用寄存器操作方式
- (3) 用固件库函数操作方式

完成如下功能：初始化端口，起始时指示灯LED1~LED4全灭。当按下KEY1时让LED1和LED3亮，LED2和LED4灭，按下KEY2让LED1和LED3灭，LED2和LED4亮的程序片段。

已知PD端口地址为：

GPIOD_CRL: 0x40011400
GPIOD_CRH: 0x40011404
GPIOD_IDR: 0x40011408
GPIOD_ODR: 0x4001140C



(1) 汇编

```
GPIOD_CRL EQU 0x40011400
GPIOD_CRH EQU 0x40011404
GPIOD_IDR EQU 0x40011408
GPIOD_ODR EQU 0x4001140C
```

```
LDR R0,=GPIOD_CRL
LDR R1,[R0]
LDR R2,=0xFFFF
ANDS R1,R1,R2
STR R1,[R0]
LDR R0,=GPIOD_CRH
LDR R1,[R0]
```

按图示连接，分别用

(1) 用汇编语言操作方式

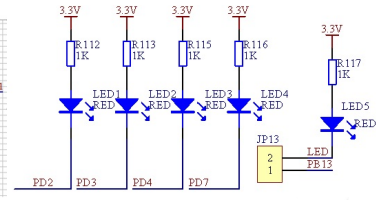
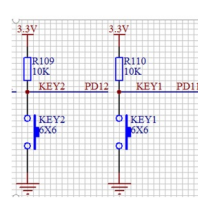
(2) 用寄存器操作方式

(3) 用固件库函数操作方式

完成如下功能：初始化端口，起始时指示灯LED1~LED4全灭。当按下KEY1时让LED1和LED3亮，LED2和LED4灭，按下KEY2让LED1和LED3灭，LED2和LED4亮的程序片段。

已知PD端口地址为：

GPIOD_CRL: 0x40011400
GPIOD_CRH: 0x40011404
GPIOD_IDR: 0x40011408
GPIOD_ODR: 0x4001140C



```

LDR    R2, =0xFFF44FFF
ANDS   R1, R1, R2
LDR    R2, =0x00044000
ORRS   R1, R1, R2
STR    R1, [R0]
MOV    R2, #0

L0:    LDR    R0, =GPIOD_IDR
        LDR    R1, [R0]
        LDR    R2, =0x00000800
        ANDS   R1, R2
        BEQ    L1
        LDR    R2, =0x0000/000
        ANDS   R1, R2
        BEQ    L2
        MOV    R2, #1
        B      L3
L2:    MOV    R2, #2
L3:    LDR    R0, =GPIOD_ODR
        CMP    R2, #1
        BEQ    LED13
        CMP    R2, #2
        BEQ    LED24
        B      LEDCOM
LED13: LDR    R1, [R0]
        LDR    R2, =0xFFFFFEB
        ANDS   R1, R1, R2
        LDR    R2, =0x00000088
        ORRS   R1, R1, R2
        STR    R1, [R0]
        B      LEDCOM
LED24: LDR    R1, [R0]
        LDR    R2, =0xFFFFF77
        ANDS   R1, R1, R2
        ORRS   R1, R1, #0x14
        STR    R1, [R0]
        B      LEDCOM
LEDCOM B      L0
    
```

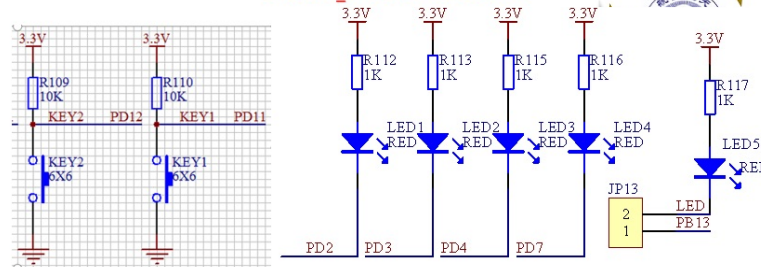
按图示连接，分别用

- (1) 用汇编语言操作方式
- (2) 用寄存器操作方式
- (3) 用固件库函数操作方式

完成如下功能：初始化端口，起始时指示灯LED1~LED4全灭。当按下KEY1时让LED1和LED3亮，LED2和LED4灭，按下KEY2让LED1和LED3灭，LED2和LED4亮的程序片段。

已知PD端口地址为：

GPIOD_CRL: 0x40011400
GPIOD_CRH: 0x40011404
GPIOD_IDR: 0x40011408
GPIOD_ODR: 0x4001140C



(2) 寄存器

```
GPIOD->CRL &= ~(0xE << 8) & ~(0xE << 12) & /  
~(0xE << 16);
```

```
GPIOD->CRL |= (4 << 12) | (4 << 16);
```

```
GPIOD->CRL &= ~(0xB << 12) & ~(0xB << 16);
```

```
while (1)
```

```
if (GPIOD->IDR & (1 << 11) == 0)
```

```
{
```

```
    GPIOD->ODR &= ~(1 << 2) & ~(1 << 4);
```

```
    GPIOD->ODR |= (1 << 3) | (1 << 7);
```

```
}
```

```
else if (GPIOD->IDR & (1 << 12) == 0)
```

```
{
```

```
    GPIOD->ODR &= ~(1 << 3) & ~(1 << 7);
```

```
    GPIOD->ODR |= (1 << 2) | (1 << 4);
```

```
}
```

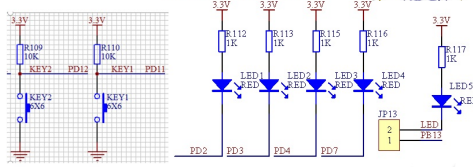
按图连接，分别用

- (1) 用汇编语言操作方式
- (2) 用寄存器操作方式
- (3) 用固件库函数操作方式

完成如下功能：初始化端口，起始时指示灯LED1~LED4全灭。当按下KEY1时让LED1和LED3亮，LED2和LED4灭，按下KEY2让LED1和LED3灭，LED2和LED4亮的程序片段。

已知PD端口地址为：

GPIO_CRL: 0x40011400
GPIO_CRH: 0x40011404
GPIO_IDR: 0x40011408
GPIO_ODR: 0x4001140C



(3) 固件库函数

SystemInit();

GPIO_InitTypeDef GPIO_InitStructure;

RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOD, ENABLE);

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_11 | GPIO_Pin_12;

GPIO_InitStructure.GPIO_Speed = GPIO_Speed_2MHz;

GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;

GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2 | GPIO_Pin_3 | /
GPIO_Pin_4 | GPIO_Pin_7;

GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT_PP;

GPIO_Init(GPIOD, &GPIO_InitStructure);

GPIO_SetBits(GPIOD, GPIO_Pin_2 | GPIO_Pin_3 | /
GPIO_Pin_4 | GPIO_Pin_7);

while (1)

if (GPIO_ReadInputDataBit(GPIO_Pin_11) == 0)
{

GPIO_SetBits(GPIOD, GPIO_Pin_3 | GPIO_Pin_7);

GPIO_ResetBits(GPIOD, GPIO_Pin_2 | GPIO_Pin_4);

}

else if (GPIO_ReadInputDataBit(GPIO_Pin_12) == 0)
{

GPIO_SetBits(GPIOD, GPIO_Pin_3 | GPIO_Pin_7);

GPIO_ResetBits(GPIOD, GPIO_Pin_2 | GPIO_Pin_4);

}